

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 7

Three Small Programs: Processes, Redirection, a Pipe

Coșneanu Corina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 LABORATORY OBJECTIVES AND THEORY **1**
1.1 Objective 1
1.2 Theoretical Background 1

2 PRACTICAL WORK AND OBSERVATIONS **2**
2.1 Part 1: Process Creation with fork, exec, and wait 2
2.1.1 Observe Answers (Part 1) 2
2.2 Part 2: Zombie Process Creation 3
2.2.1 Observe Answers (Part 2) 3
2.3 Part 3: Output Redirection 4
2.3.1 Observe Answers (Part 3) 4
2.4 Part 4: Pipeline Implementation 5
2.4.1 Observe Answers (Part 4) 5

3 CONCLUSIONS **6**

1 LABORATORY OBJECTIVES AND THEORY

1.1 Objective

The main objective of this lab session is to investigate POSIX-based process control within Linux environments. This includes spawning new processes with `fork()`, swapping execution images via `execlp()`, coordinating process termination through `waitpid()`, observing zombie states, and handling standard file descriptors to enable I/O redirection and pipe-based inter-process communication.

1.2 Theoretical Background

Within UNIX-based architectures, the generation of processes and management of I/O rely on core POSIX concepts:

- **Process Generation (`fork` + `exec`):** A fresh process emerges by duplicating the parent utilizing `fork()`. Afterwards, the child can call `exec()` to replace its current memory space with a distinct executable binary.
- **Parent Coordination & Zombie Status:** If a child finishes its task via `exit()`, it lingers in the system's process table as a Zombie (labeled as `Z`) until the parent explicitly collects its termination status using `wait()` or `waitpid()`.
- **File Descriptors & I/O Routing:** All processes inherit a standard set of descriptors: `0` (stdin), `1` (stdout), and `2` (stderr). Functions like `open()` will systematically assign the smallest available integer as the new descriptor.
- **Pipe-Based Communication:** Generating a pipe via `pipe()` creates a one-way communication channel, featuring a reading end (`p[0]`) alongside a writing end (`p[1]`).

2 PRACTICAL WORK AND OBSERVATIONS

2.1 Part 1: Process Creation with fork, exec, and wait

The `run.c` application illustrates the behavior of a basic shell: it duplicates itself, substitutes the child process's memory with the `ls` command, and pauses until the execution finishes.

```
vboxuser@lab1:~/os-lab7$ nano run.c
vboxuser@lab1:~/os-lab7$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 15946
child:  my PID is 15947, my parent is 15946
total 24
-rwxrwxr-x 1 vboxuser vboxuser 16392 Oct 19 13:23 run
-rw-rw-r-- 1 vboxuser vboxuser   601 Oct 19 13:22 run.c
parent: child 15947 ended with status 0
vboxuser@lab1:~/os-lab7$ nano run.c
vboxuser@lab1:~/os-lab7$ gcc -Wall -Wextra -o run run.c && ./run
parent: my PID is 15991
child:  my PID is 15992, my parent is 15991
exec: No such file or directory
parent: child 15992 ended with status 127
```

Figure 2.1: Execution of run.c with successful and failing exec calls

2.1.1 Observe Answers (Part 1)

1. **Reaching `perror("exec")`:** Because a successful `execlp()` call entirely replaces the child's memory image, the lines following it will never run. Therefore, `perror("exec")` is triggered exclusively if the execution fails (for instance, when trying to run `nosuchprog`).

2. **Recognizing the Parent Process:** The newly created child displays the PID of its creator (the process that executed `fork()`), which is retrieved using the `getppid()` function.

3. **Evaluating Exit Codes:** The parent process detects a status code of `0` when running valid commands (like `ls` or `date`) and receives `127` when attempting to run invalid or missing programs.

2.2 Part 2: Zombie Process Creation

By forcing the parent to wait via a `sleep(20)` before calling `waitpid()`, the child process is intentionally forced into a zombie state right after it finishes.

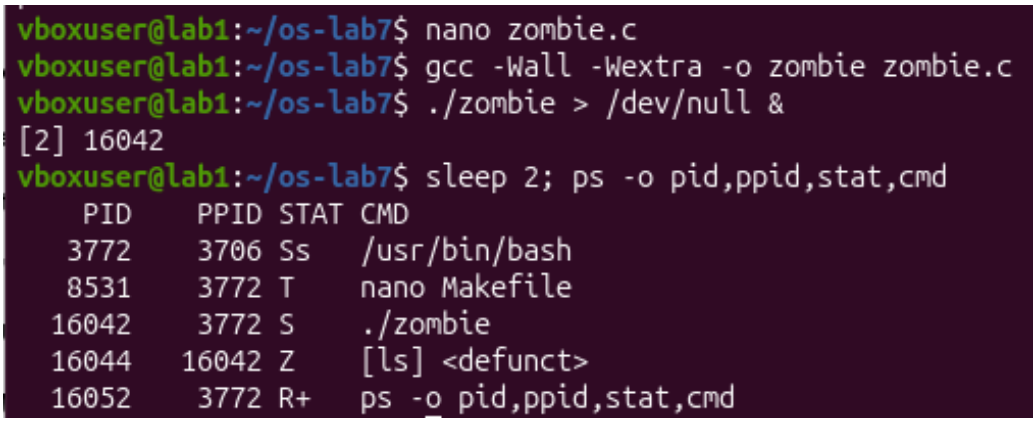


Figure 2.2: Process table inspection showing zombie process state

2.2.1 Observe Answers (Part 2)

1. **Identifying the Process State:** The presence of `Z` (or `Z+`) under the `STAT` column denotes a defunct (Zombie) process. This means the program finished its work but is still waiting for its parent to acknowledge its termination.

2. **Eliminating the Zombie:** Once the 20-second delay passes, the `waitpid(pid, &status, 0)` call successfully retrieves the exit code, which clears the zombie from the system's process table.

2.3 Part 3: Output Redirection

Closing the standard output descriptor (`1`) prior to opening `out.txt` successfully diverts all output directly to a file on the disk.

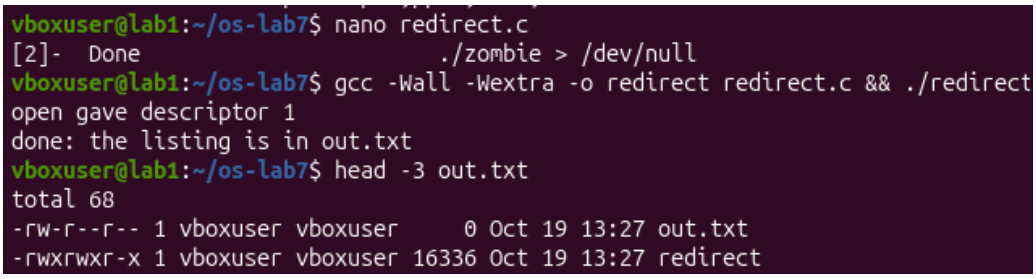


Figure 2.3: Output redirection execution and file inspection

2.3.1 Observe Answers (Part 3)

1. **Assigning the Descriptor:** The `open()` function returned descriptor `1` because the OS dictates that the smallest unused integer must be allocated. Since we manually closed `1`, it became the first available slot.

2. **Absence of Terminal Output:** Nothing is printed to the screen because the standard output pointer (`1`) is now securely linked to the `out.txt` file instead of the console.

3. **Redirecting Standard Error:** If we wanted to route error logs to a file (`2> err.txt`), we would simply need to close descriptor `2` (`stderr`) right before executing the `open()` command.

2.4 Part 4: Pipeline Implementation

Data transfer between the `ls /etc` command and `wc -l` is facilitated through the implementation of a pipeline.

```
vboxuser@lab1:~/os-lab7$ nano pipeline.c
vboxuser@lab1:~/os-lab7$ gcc -Wall -Wextra -o pipeline pipeline.c && ./pipeline
236
vboxuser@lab1:~/os-lab7$ ls /etc | wc -l
236
```

Figure 2.4: Pipeline execution comparing custom C program and shell output

2.4.1 Observe Answers (Part 4)

1. **Confirming Pipeline Accuracy:** The output generated by our `./pipeline` program precisely matched the line count produced by the native shell command `ls /etc | wc -l`.

2. **Process Freezing:** If we disable the `close(p[1])` instruction within the parent code, the second child process (`wc`) becomes permanently stuck.

3. **Explaining the Blockage:** The `wc` program remains paused because it expects an End-Of-File (EOF) marker on `p[0]`. This marker is never delivered as long as the parent process keeps the write end of the pipe (`p[1]`) open.

3 CONCLUSIONS

Throughout this lab assignment, I acquired hands-on skills regarding POSIX process handling, synchronization techniques, and the management of file descriptors in a Linux environment. By writing `run.c`, I observed the foundational mechanics of how a shell operates, relying on the `fork()`, `exec()`, and `waitpid()` sequence.

Additionally, the analysis of defunct processes in `zombie.c` emphasized the importance of proper process table maintenance at the kernel level. Finally, experimenting with descriptors in `redirect.c` and `pipeline.c` provided a clear view of how UNIX-like systems perform I/O redirection and enable inter-process communication through the strategic reassignment of low-level file descriptors.